

PIC

Design Specification

Programmable interrupt controller · interface and register contract

Document	PIC — Design Specification
Document type	Design specification
Subject	Programmable interrupt controller · interface and register contract
Revision	2.2
Date	23 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	2
1.1	Purpose of the document	2
1.2	Validity of the document	2
1.3	Definitions of terms and abbreviations	2
1.4	Relationship with other documents	2
1.5	Notes on the notation used	3
2	Specifications and system integration	4
2.1	Requirements and integration	4
2.2	Configuration and integration limits	4
2.3	Interface adaptations	5
3	Overview of overall structure	6
3.1	Arbitration and service contract	7
3.2	Source aggregation and offer timing	7
3.3	Spurious and deadline policy	7
3.4	Level request, claim and EOI	8
3.5	Preemption and nesting	9
3.6	Edge capture and spurious claims	9
3.7	Deadline escalation	10
4	Interfaces	12
4.1	Hardware interface	12
4.1.1	Reset and register-access contract	13
4.2	Software interface	13
4.2.1	SRCx_CONFIG — 0x00 + 4x	14
4.2.2	SRCx_SW_TRIG — 0x40 + 4x	14
4.2.3	SRCx_STATUS — 0x80 + 4x	15
4.2.4	BAND_CONFIG — 0xC0	15
4.2.5	NEST_STATUS — 0xC4	15
4.2.6	NEST_MAX — 0xC8	16
4.2.7	ACTIVE_VEC — 0xCC	16
4.2.8	SPURIOUS_LOG — 0xD0	16
4.2.9	ESCALATION_CFG — 0xD4	16
4.2.10	INT_ENABLE — 0xD8	17
4.2.11	INT_STATUS — 0xDC	17
4.2.12	Programming sequences	17
5	Reuse and reusability	19
6	Design of submodules	20
6.1	axi_lite_slave	20
7	List of bugs and workarounds	22

1 Introduction

1.1 Purpose of the document

This specification describes the implemented RTL, programming model and integration contracts of the programmable interrupt controller. It is intended for hardware designers, verification engineers and software developers integrating the block.

It defines the behaviour that the RTL must show at its interfaces.

1.2 Validity of the document

The documented configuration is the RTL in `cpu/hdl` and the associated CPU/PIC and SoC simulation environments. Interface adaptations from the supplied briefs are identified in the integration section.

Revision 2.2 of 23 September 2026 describes the delivered state of that RTL.

1.3 Definitions of terms and abbreviations

Term	Meaning
AXI4-Lite	Single-beat AXI read/write channels.
CSR	Control and status register.
DUT	Device under test.
EOI	End of interrupt; completes one service level.
IRQ	Interrupt request.
NBA	Nonblocking-assignment update region in simulation.
PIC	Programmable interrupt controller.
SVA	SystemVerilog Assertions.

1.4 Relationship with other documents

The documents below are the ones this specification is derived from or read with.

Document	Version / date	Author	Relationship
Programmable Interrupt Controller with Advanced Scheduling	Draft, 26 June 2026	D. Borzasi, Siemens	Functional requirements; see section 2.
Spec table of contents and templates	2 September 2026	D. Borzasi, Siemens	Document structure and verification-plan template.
Verification Specification — PIC	Rev. 2.2, 23 September 2026	C.-A. Bunea	Companion document.
Design Specification — RV32I CPU	Rev. 2.2, 23 September 2026	C.-A. Bunea	Other side of the CPU–PIC interrupt link.
AMBA AXI and ACE Protocol Specification	IHI0022E	Arm	AXI4-Lite ports.

Document	Version / date	Author	Relationship
IEEE Std 1364 (Verilog)	2005	IEEE	RTL language.

1.5 Notes on the notation used

Signal and register names follow the RTL. Addresses and waveform bus values are hexadecimal; time is in ns and widths are in bits. Register offsets are byte offsets; CSR addresses are instruction-encoded indices.

Notation	Meaning
Fixed-width name	A Verilog module, port, file or script name, spelled as in the source.
Signal suffix <code>_i</code> / <code>_o</code>	Module input / output, seen from inside that module.
<code>[a:b]</code>	Bit range, most significant bit first.
Source slot	One of the 16 arbitration identities; x or n denotes its index 0–15.
posedge +1 ns	A testbench stimulus time, one nanosecond after the sampling edge.

2 Specifications and system integration

2.1 Requirements and integration

The table sets each requirement of the brief Programmable Interrupt Controller with Advanced Scheduling against the contract implemented in `pic.v` and `axi_lite_slave.v`. In the SoC, the register port sits on the CPU data decoder; sources 0–3 are the DMA channels, 4 the SRAM and 7 the timer, and the remaining inputs are tied low.

Topic	Initial requirement	Implemented contract
Sources	16 hardware lines and 16 software-trigger channels.	HW/SW merge into 16 source slots; one vector per slot.
Priority grouping	Define bands and inter-/intra-band arbitration.	Four bands; programmable 2-bit urgency; 4-bit intra-band priority; lowest ID breaks ties.
Preemption	Preserve and restore active contexts.	16-entry stack; configured limit 1–16, reset 8; explicit EOI pops one level.
Spurious requests	Detect a source withdrawn before acknowledgement.	Claim-time detection; source and global sticky logs.
Deadlines	Per-source service deadline and automatic escalation.	16-bit cycle count; jump or next-more-urgent-band policy; optional repeated escalation.
Software trigger	Protect injection; define clear and HW/SW collision.	0xA5A5 key; key byte strobes required; zero or claim clears request.
Register port	AXI4-Lite slave; documented register map.	32-bit data/address; 256-byte window; independent AW/W collection.
Reset / target	Active-low asynchronous reset; behavioural RTL simulation.	posedge <code>clk_i</code> ; reset disables all sources; no synthesis result claimed.

2.2 Configuration and integration limits

The module has no Verilog parameters: every quantity is either fixed in the RTL or programmed through a register. The first table lists the fixed quantities; the second states what an integrator has to guarantee outside the block.

Fixed quantity	Value
Source slots	16; each merges a hardware line and software request.
Priority bands	4; configurable urgency, not additional modules.
Physical nesting depth	16; run-time <code>NEST_MAX</code> defaults to 8.
AXI width	32-bit address/data; four strobes; three protection bits; two response bits.

Constraint	Required integration behaviour
Address window	Decode exactly a 256-byte window. Only address bits [7:2] are used; upper bits alias and low bits select the containing word.
IRQ input synchrony	<code>irq_src_i</code> must be synchronous to <code>clk_i</code> ; synchronize asynchronous peripheral sources outside the PIC.
CPU handshake	Claim refers to the offer sampled one cycle earlier. Never assert claim and EOI together.

Constraint	Required integration behaviour
One CPU	One offer / mask / claim / EOI interface per PIC instance.
Protection	AWPROT/ARPROT accepted and ignored; no access-control mechanism in this block.
Implementation target	Behavioural RTL simulation. No synthesis / frequency claim.

2.3 Interface adaptations

The integrated CPU/PIC interface resolves the different source counts in the two drafts. The following choices define the implemented connection and arbitration timing.

Brief item	Implemented contract
CPU brief IRQ interface	8 request bits / 3-bit ID / 8 acknowledge bits become one offer, 4-bit source ID, 16-bit pending/mask and scalar claim/EOI.
Claim / completion	Claim starts active service and consumes latched requests. The added EOI input completes service and pops one context.
Resolution pipeline	The PIC brief describes staged resolution. This RTL resolves eligibility and the priority key combinationally and registers the offer; there are no successive registered arbitration stages.

3 Overview of overall structure

An enabled HW or software request enters priority selection. The registered offer identifies the candidate; claim saves its ID and priority, while EOI restores the previous active context. The return paths carry masks, the saved priority threshold and the depth limit.

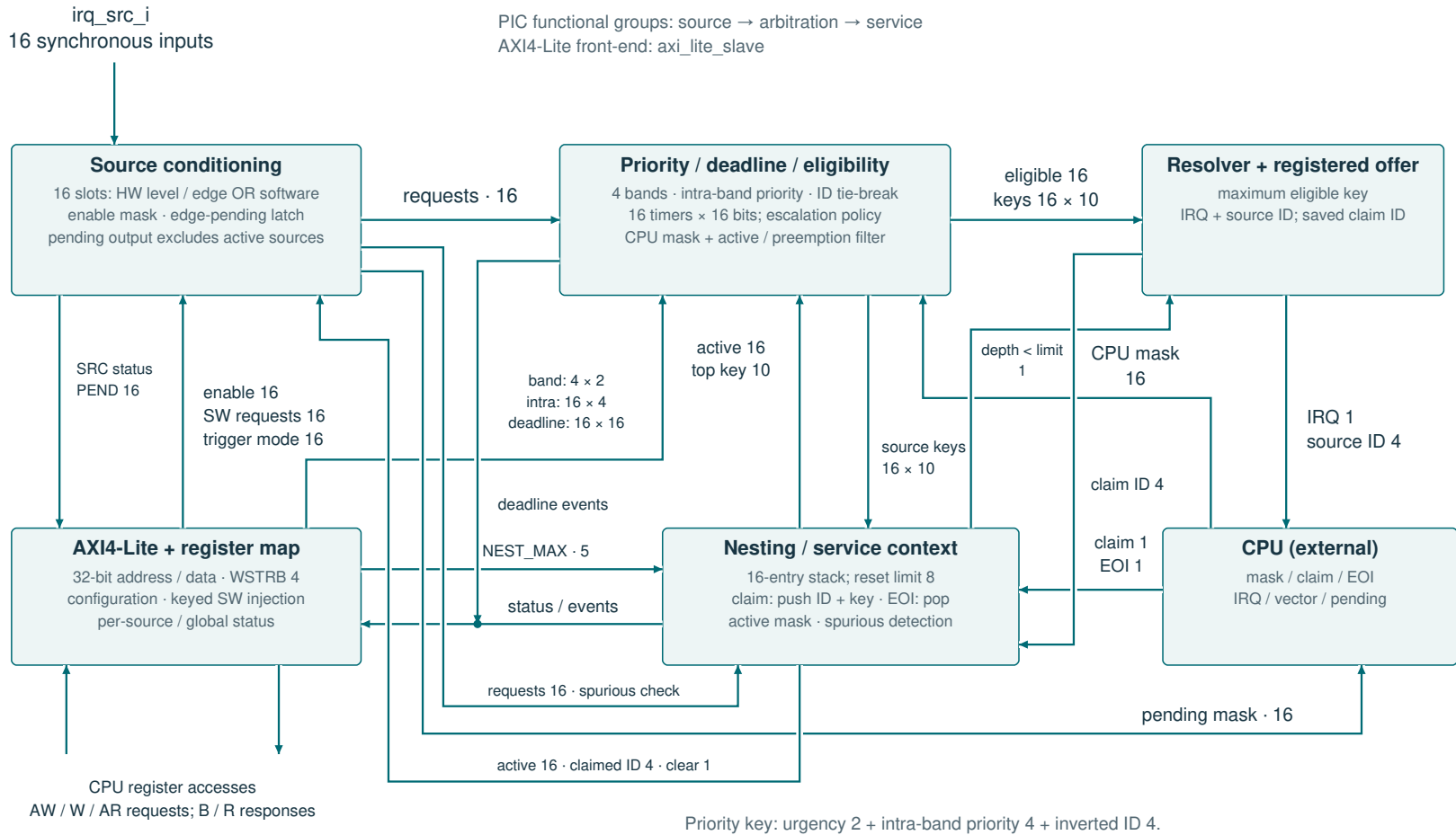


Figure 1: Architectural view: HW/SW requests share 16 source slots; the stack records active service contexts.

3.1 Arbitration and service contract

Arbitration answers one question per cycle: which enabled source, if any, may be offered to the CPU. The table fixes how the winner is chosen, when a candidate may preempt an active context, and what the claim records.

Decision	Contract
Priority key	Band urgency first, then higher intra-band priority, then lower source ID.
Preemption	A candidate must be enabled, CPU-unmasked, inactive and strictly more urgent than the active top context.
Recorded context	Claim snapshots the selected source ID and priority. Reconfiguring an active source does not change that saved threshold.
Offer	cpu_irq_o and cpu_irq_vec_o are registered together; pending_o is a separate combinational mask.

3.2 Source aggregation and offer timing

The controller maintains one service identity per source slot. HW and SW requests for the same slot share configuration, priority, deadline and active state.

Decision	Behaviour
HW / SW collision	The enabled request is (selected HW level/latched edge OR SW pending). Simultaneous HW and SW activity produces one candidate, not two queued interrupt identities.
Pending storage	Edge and software requests use one pending bit per slot; repeated events before consumption coalesce. Level sources remain pending while the external line remains asserted.
Same-cycle events	A fresh edge wins over clearing the previous edge on claim. A keyed software set wins over the claim clear. After EOI, a retained request may be offered again.
Priority key	Four configurable 2-bit band urgencies, a per-source 4-bit intra-band priority, then inverted 4-bit ID. Select the largest key; lower source ID wins the final tie.
Reconfiguration	Changing an active source configuration affects future arbitration; the active stack entry keeps the key captured at claim until that context is popped.
Offer latency	Request conditioning and key comparison are combinational; IRQ/vector are registered at posedge. The claim consumes a one-cycle delayed copy of the offered vector.
Depth enforcement	Only an inactive, enabled, CPU-unmasked source above the saved top key is eligible. <code>depth < NEST_MAX</code> is additionally required for an offer; an eligible blocked source sets OVF.

3.3 Spurious and deadline policy

The service decision uses the request state at claim. Deadline accounting covers enabled requests waiting for service, including requests masked at the CPU.

Decision	Behaviour
Spurious criterion	A claim is spurious when its saved source ID is no longer requested. Preserve the claimed service context, mark SRCx_STATUS.SPUR, and set SPURIOUS_LOG and INT_STATUS.SPUR; EOI unwinds it normally.
Fast-clearing source	A level pulse gone before claim meets the same spurious criterion. Edge mode latches a pulse until claim and is the appropriate configuration when the event must survive deassertion.
Deadline storage / count	SRCx_CONFIG[31:16] stores the 16-bit cycle deadline. Each slot counts while enabled pending and inactive; zero disables counting. CPU masking does not pause the timer.
Escalation	At deadline, either jump to the configured target band or select the next strictly greater programmed urgency. MULTI enables another escalation at each further deadline; the effective band cannot advance beyond the highest urgency.
After service / idle	On the first edge observing no wait, clear the counter and wait-local escalation flag and restore the configured band. Sticky INT_STATUS.ESC remains until software clears it.
Software injection	Write 0xA5A50001 to SRCx_SW_TRIG with WSTRB[3:2]=11 and WSTRB[0]=1. Clear explicitly with bit 0=0, or automatically by claim. A software-only request follows the same deadline policy as hardware.

3.4 Level request, claim and EOI

Source 3 is offered and claimed once. Claim increases the active depth; after the source clears, EOI returns the controller to depth zero.

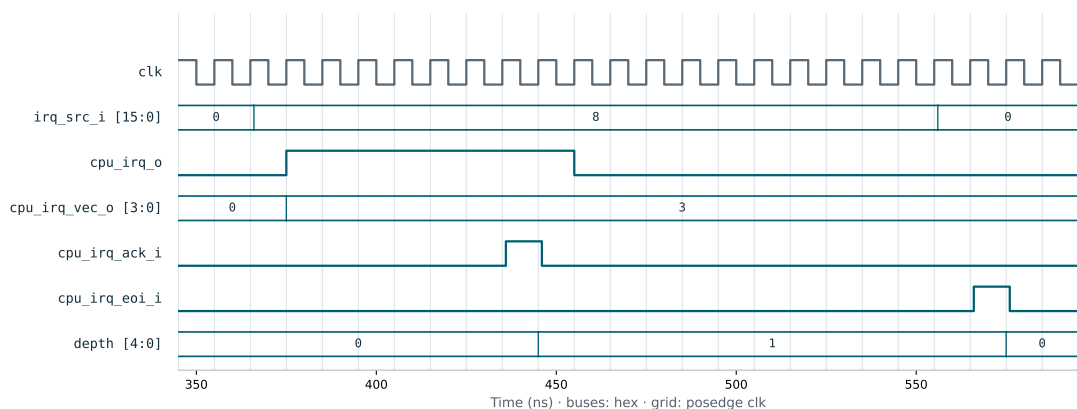


Figure 2: Source 3 service: the request is claimed once; EOI returns nesting depth to zero.

Condition	Observable behaviour
Enabled level high	Source becomes pending. cpu_irq_o is registered on the next resolution.
Claim	Push the previously offered source; mark it active; active sources are excluded from offers.
Source clears	The handler removes the peripheral cause before return.
EOI	Pop exactly one level. EOI on an empty stack is ignored.
Source remains high after EOI	It can become eligible again; PIC does not clear the external peripheral.

3.5 Preemption and nesting

Source 9 preempts source 8. The active mask keeps both sources during nesting; the first EOI restores source 8 and the second empties the stack. The offer vector is meaningful while `cpu_irq_o` is high.

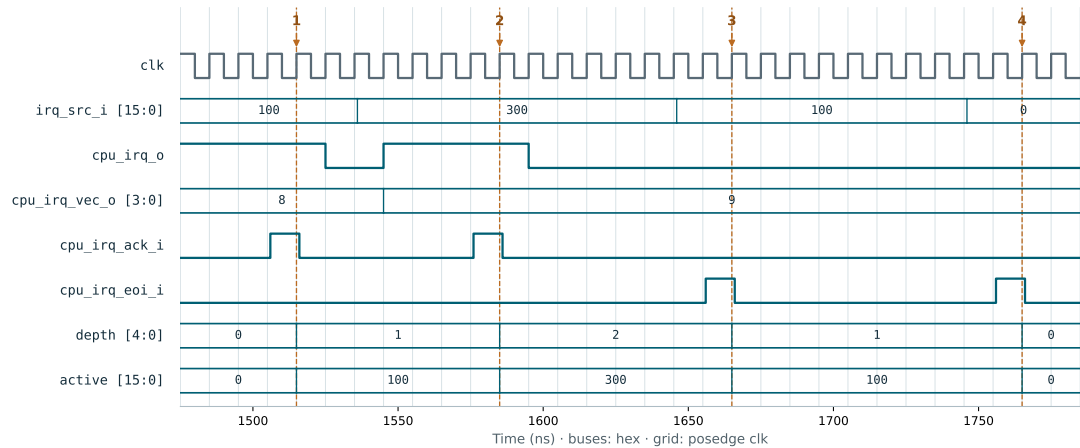


Figure 3: Source 8 enters service, source 9 preempts, and two EOIs restore then remove the outer context.

Condition	Result
Higher priority arrives	It may be offered while the current source is active.
Second claim	Depth increases from 1 to 2; outer source remains active.
Inner EOI	Depth returns to 1; outer priority threshold is restored.
At NEST_MAX	No new offer; an eligible blocked source sets OVF.
Reband active source	Saved priority threshold is unchanged until that context is popped.

3.6 Edge capture and spurious claims

A second source edge is sampled with the claim of the first event. The edge latch remains set, and the source is offered again after EOI.

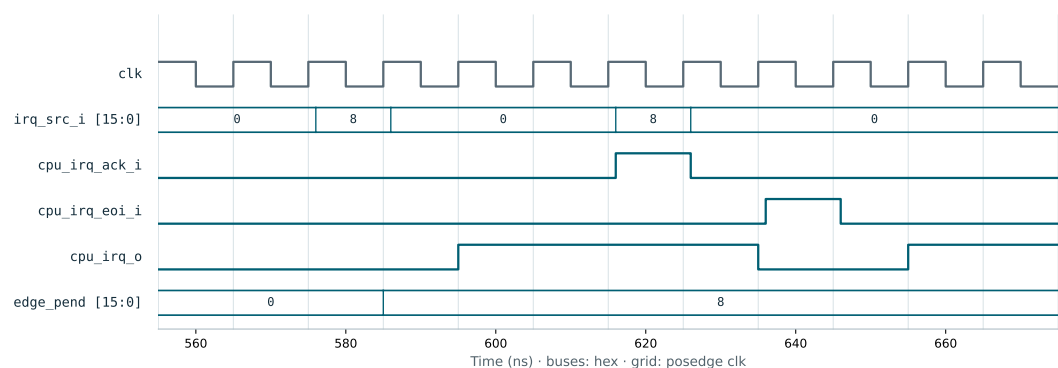


Figure 4: Simultaneous claim and a new source edge: the new edge remains latched for service after EOI.

The level source disappears after its offer but before claim. The claim creates a service context marked spurious, allowing the CPU to unwind it through the same EOI sequence.

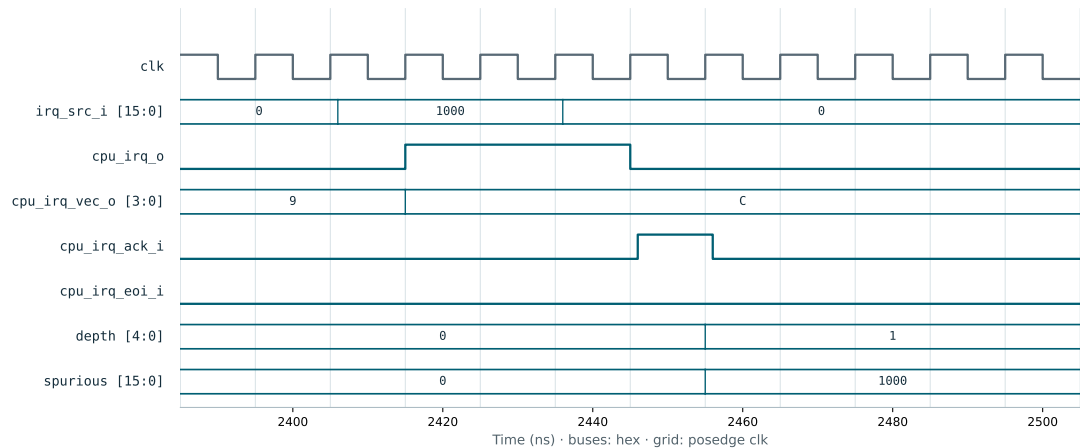


Figure 5: Source withdrawal before claim: the active context is marked spurious and the event is logged.

Event	Contract
Disabled edge source	A rising edge remains latched; enabling later exposes it. Level-mode reset slots capture no edges.
Claim with a new edge	New edge takes precedence over consuming the previous pending edge.
Spurious claim	The previously offered source is no longer pending when claimed; log globally and per source; unwind with normal EOI.

3.7 Deadline escalation

Source 1 initially wins the tie. Source 2 reaches its ten-cycle deadline; the escalation event changes its effective priority, and the registered offer changes to source 2 on the following edge.

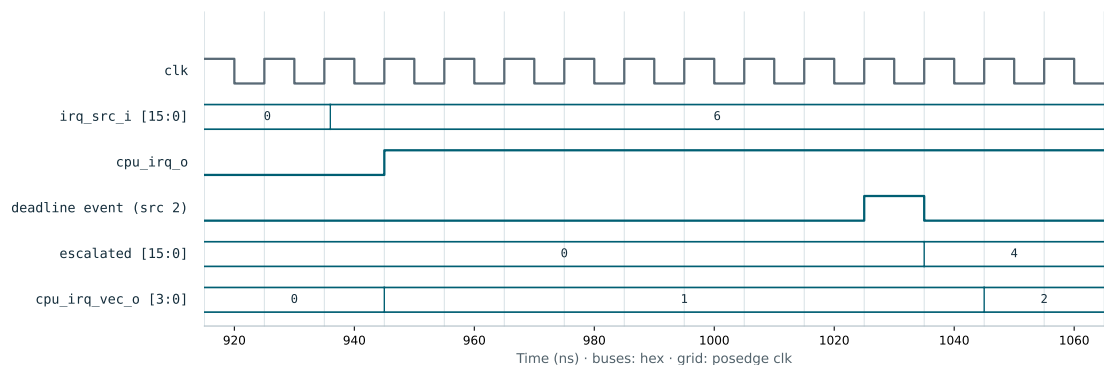


Figure 6: Reordered bands, source 2 deadline = 10 cycles: escalation precedes the registered offer changing from source 1 to 2.

Setting / event	Behaviour
DEADLINE=0	Timer disabled and held at zero.
Nonzero deadline	Count while enabled pending and not active. CPU masking does not stop the pending timer.
MODE=0	Jump to TARGET when deadline expires.
MODE=1	Move to the next strictly greater urgency in BAND_CONFIG, which need not be the next lower band number.

Setting / event	Behaviour
MULTI=1	Repeat at each deadline. In bump mode, the effective band stays unchanged if no higher urgency exists.
Claim / no longer pending	Clear elapsed wait and escalation state; configured priority applies again.
Status	SRCx_STATUS.ESC is wait-local; INT_STATUS.ESC is sticky until RW1C.

4 Interfaces

4.1 Hardware interface

All ports are digital and synchronous to the rising edge of `clk_i`; `rst_n_i` is the only asynchronous input. The ports are the 16 source inputs, the CPU handshake (offer to the CPU, claim and EOI back) and the AXI4-Lite register port.

Signal	Dir.	Width (bits)	Meaning
<code>clk_i</code>	input	1	Rising-edge clock input.
<code>rst_n_i</code>	input	1	Asynchronous reset, active low.
<code>irq_src_i</code>	input	16	16 synchronous hardware interrupt source lines.
<code>cpu_mask_i</code>	input	16	CPU acceptance mask, applied before priority selection.
<code>cpu_irq_o</code>	output	1	Registered interrupt offer; vector is valid while high.
<code>cpu_irq_vec_o</code>	output	4	Registered offered source ID; meaningful while <code>cpu_irq_o</code> is high.
<code>pending_o</code>	output	16	Enabled pending sources excluding active sources; independent of CPU mask.
<code>cpu_irq_ack_i</code>	input	1	One-cycle claim of the previously sampled offer.
<code>cpu_irq_eoi_i</code>	input	1	One-cycle end of interrupt; pop one active level.
<code>s_axi_awaddr_i</code>	input	32	Write byte address.
<code>s_axi_awprot_i</code>	input	3	Write protection attributes.
<code>s_axi_awvalid_i</code>	input	1	Write address is valid; hold until handshake.
<code>s_axi_awready_o</code>	output	1	Receiver can accept the write address.
<code>s_axi_wdata_i</code>	input	32	Write payload.
<code>s_axi_wstrb_i</code>	input	4	One enable bit per write byte.
<code>s_axi_wvalid_i</code>	input	1	Write data is valid; hold until handshake.
<code>s_axi_wready_o</code>	output	1	Receiver can accept the write data.
<code>s_axi_bresp_o</code>	output	2	Write response: OKAY=00, SLVERR=10, DECERR=11.
<code>s_axi_bvalid_o</code>	output	1	Write response is valid; hold until handshake.
<code>s_axi_bready_i</code>	input	1	Receiver can accept the write response.
<code>s_axi_araddr_i</code>	input	32	Read byte address.
<code>s_axi_arprot_i</code>	input	3	Read protection attributes.
<code>s_axi_arvalid_i</code>	input	1	Read address is valid; hold until handshake.
<code>s_axi_arready_o</code>	output	1	Receiver can accept the read address.
<code>s_axi_rdata_o</code>	output	32	Read payload.

Signal	Dir.	Width (bits)	Meaning
s_axi_rresp_o	output	2	Read response: OKAY=00, SLVERR=10, DECERR=11.
s_axi_rvalid_o	output	1	Read response is valid; hold until handshake.
s_axi_rready_i	input	1	Receiver can accept the read response.

4.1.1 Reset and register-access contract

The controller starts with active nested service and enabled sources. Reset clears both depth and INT_ENABLE between clock edges; the test checks the state one nanosecond after assertion.

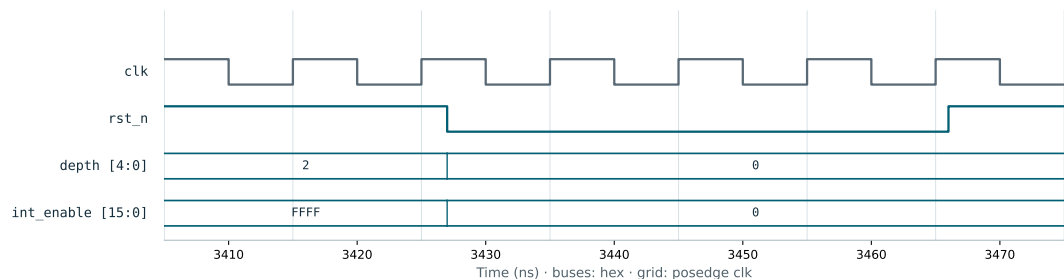


Figure 7: Reset from a configured state: rst_n asserts at posedge +2 ns; depth and enables clear before the next edge.

Item	Behaviour
Reset	Asynchronous active-low rst_n_i; sequential state updates on posedge clk_i.
Reset state	INT_ENABLE=0, depth=0, all source requests inactive; BAND_CONFIG=0x1B, NEST_MAX=8; other readable state zero.
Transactions	One outstanding read and one outstanding write. AW and W may arrive independently, in either order.
Read / write response	OKAY for a mapped permitted access. SLVERR for unmapped read/write or a write to RO. Rejected read data is zero.
Writes	WSTRB applies per byte as described in each field. No strobe does not authorize an otherwise forbidden write.
Reset during transfer	Captured AW/W and pending B/R responses are cleared; no pre-reset response is replayed afterwards.

4.2 Software interface

Every software-visible control and status bit of the controller lives in one 256-byte AXI4-Lite window. The map lists each offset with its access class and reset value; the subsections that follow give the fields of each register.

Byte offset	Name	Access type	Reset	Description
0x00–0x3C	SRC0..15_CONFIG	RW	0x00000000	Trigger, band, priority, deadline.

Byte offset	Name	Access type	Reset	Description
0x40–0x7C	SRC0..15_SW_TRIG	RW	0x00000000	Keyed injection / clear.
0x80–0xBC	SRC0..15_STATUS	RO	0x00000000	Per-source state and timer.
0xC0	BAND_CONFIG	RW	0x0000001B	Four urgency values.
0xC4	NEST_STATUS	RO	0x00000000	Depth, top ID and overflow.
0xC8	NEST_MAX	RW / WARL	0x00000008	Allowed active depth.
0xCC	ACTIVE_VEC	RO	0x00000000	Active source and valid bit.
0xD0	SPURIOUS_LOG	RW1C	0x00000000	Per-source spurious flags.
0xD4	ESCALATION_CFG	RW	0x00000000	Escalation policy.
0xD8	INT_ENABLE	RW	0x00000000	Source enables.
0xDC	INT_STATUS	RW1C	0x00000000	Global event flags.

Rule	Meaning
Per-source offsets	x ranges from 0 to 15. Configuration=4x; software trigger=0x40+4x; status=0x80+4x.
Reserved addresses	0xE0–0xFC return SLVERR.
Access classes	RW: read/write; RO: read-only; RW1C: write-one-to-clear; WARL: value coerced to a legal setting; RAZ/WI: read zero / write ignored.

4.2.1 SRCx_CONFIG — 0x00 + 4x

Bits	Field	Access type	Reset	Description
[31:16]	DEADLINE	RW	0	Pending cycles to escalation; zero disables the timer.
[15:8]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[7:4]	INTRA	RW	0	Higher value means higher priority within equal band urgency.
[3]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[2:1]	BAND	RW	0	Band index 0–3; urgency comes from BAND_CONFIG.
[0]	TRIG	RW	0	0: level. 1: latch rising edges until claim.

4.2.2 SRCx_SW_TRIG — 0x40 + 4x

Bits	Field	Access type	Reset	Description
[31:16]	KEY	WO / RAZ	0	0xA5A5 plus WSTRB[3:2]=11 is required to set SWREQ.
[15:1]	Reserved	RAZ/WI	0	Reads zero; writes ignored.

Bits	Field	Access type	Reset	Description
[0]	SWREQ	RW + HW clear	0	Set with key; zero clears without key. WSTRB[0] required. Claim clears; a simultaneous new set is retained.

4.2.3 SRCx_STATUS — 0x80 + 4x

Bits	Field	Access type	Reset	Description
[31:16]	DDL_TIMER	RO	0	Elapsed enabled-pending cycles. Clears when idle or escalating; otherwise saturates at DEADLINE-1.
[15:6]	Reserved	RO	0	Reads zero.
[5:4]	EFF_BAND	RO	0	Configured band, or temporary escalated band.
[3]	SPUR	RO	0	This active service level was claimed after its source disappeared.
[2]	ESC	RO	0	Escalated during the present wait; clears when claimed / no longer pending.
[1]	ACTIVE	RO	0	Source occupies a nesting level.
[0]	PEND	RO	0	Enabled HW/SW request, including a held level during its handler.

4.2.4 BAND_CONFIG — 0xC0

Bits	Field	Access type	Reset	Description
[31:8]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[7:6]	BAND3_URG	RW	0	Urgency of band 3.
[5:4]	BAND2_URG	RW	1	Urgency of band 2.
[3:2]	BAND1_URG	RW	2	Urgency of band 1.
[1:0]	BAND0_URG	RW	3	Urgency of band 0. Higher urgency wins.

4.2.5 NEST_STATUS — 0xC4

Bits	Field	Access type	Reset	Description
[31:17]	Reserved	RO	0	Reads zero.
[16]	OVF	RO	0	Alias of INT_STATUS.OVF; clear at 0xDC.
[15:12]	Reserved	RO	0	Reads zero.
[11:8]	TOP_ID	RO	0	Top active source ID; zero when depth is zero.

Bits	Field	Access type	Reset	Description
[7:5]	Reserved	RO	0	Reads zero.
[4:0]	DEPTH	RO	0	Active nesting levels, 0–16.

4.2.6 NEST_MAX — 0xC8

Bits	Field	Access type	Reset	Description
[31:5]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[4:0]	LIMIT	RW / WARL	8	Clamp 0 to 1 and values above 16 to 16. Requires WSTRB[0]. Lowering the limit does not unwind active handlers.

4.2.7 ACTIVE_VEC — 0xCC

Bits	Field	Access type	Reset	Description
[31:9]	Reserved	RO	0	Reads zero.
[8]	VALID	RO	0	At least one source is in service.
[7:4]	Reserved	RO	0	Reads zero.
[3:0]	ID	RO	0	Top active source; meaningful when VALID=1.

4.2.8 SPURIOUS_LOG — 0xD0

Bits	Field	Access type	Reset	Description
[31:16]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[15:0]	FLAGS	RW1C	0	One sticky bit per spurious source. WSTRB[1:0] qualify clear. New hardware set wins over clear.

4.2.9 ESCALATION_CFG — 0xD4

Bits	Field	Access type	Reset	Description
[31:9]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[8]	MULTI	RW	0	1: repeat each deadline. 0: one escalation per wait.
[7:5]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[4]	MODE	RW	0	0: jump to TARGET. 1: next strictly more urgent band.

Bits	Field	Access type	Reset	Description
[3:2]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[1:0]	TARGET	RW	0	Destination band for jump mode.

4.2.10 INT_ENABLE — 0xD8

Bits	Field	Access type	Reset	Description
[31:16]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[15:0]	ENABLE	RW	0	One source enable per bit; byte strobes apply. CPU mask is a separate filter.

4.2.11 INT_STATUS — 0xDC

Bits	Field	Access type	Reset	Description
[31:3]	Reserved	RAZ/WI	0	Reads zero; writes ignored.
[2]	OVF	RW1C	0	Eligible source blocked by depth limit.
[1]	ESC	RW1C	0	At least one deadline escalation occurred.
[0]	SPUR	RW1C	0	At least one spurious claim occurred. New hardware event wins over a same-cycle clear.

4.2.12 Programming sequences

A source is configured while it is disabled, enabled once its policy is set, then claimed and completed by the handler. The second table names the registers that make that state observable while the system is running.

Step	Register / signal	Action
1	INT_ENABLE (0xD8)	Disable target source while configuring its trigger, band, intra priority and deadline.
2	SRCx_CONFIG (4x)	Write desired fields; set reserved bits to zero.
3	BAND_CONFIG / ESCALATION_CFG / NEST_MAX	Program shared policy as needed.
4	INT_ENABLE and cpu_mask_i	Enable source in PIC and permit it at CPU.
5	SRCx_SW_TRIG (0x40+4x)	Set: WDATA=0xA5A50001 with WSTRB=0xD or 0xF. Clear: bit 0=0 with WSTRB[0]=1.
6	cpu_irq_ack_i	Claim one cycle after observing the offered request/vector.
7	Peripheral / cpu_irq_eoi_i	Clear the cause, then issue one EOI when the handler returns.

Step	Register / signal	Action
8	INT_STATUS / SPURIOUS_LOG	Read events and write ones to the flags that should be cleared.
Useful observation		Read
Pending / active / elapsed		SRCx_STATUS.
Current handler		ACTIVE_VEC.VALID and ID.
Nested contexts		NEST_STATUS.DEPTH and TOP_ID; software cannot read the whole stack.

5 Reuse and reusability

The one reused component is the register front end `axi_lite_slave` (section 6.1), which the machine timer also uses. Reusing the controller:

Task	How to use this block
Compile	Include <code>pic.v</code> and <code>axi_lite_slave.v</code> .
Instantiate	<code>pic</code> has no Verilog module parameters; all configuration uses registers.
Connect	Map its 256-byte AXI window and connect 16 synchronous IRQ lines. Tie unused lines low.
CPU link	Connect request 1, vector 4, mask/pending 16 and claim/EOI 1 each. The original CPU brief differs; use the interface documented here.
Reset	Reset PIC, master and fabric together; keep a consistent reset domain for in-flight transactions.
Verify	Run the PIC benches in <code>regress.do</code> and the bound-assertion flow <code>run_verilator.sh</code> ; check the final address map.

6 Design of submodules

Submodule ports are synchronous to the rising edge of `clk_i` and `rst_n_i` resets asynchronously; combinational modules have neither.

6.1 axi_lite_slave

Collects independent AW/W channels and maps register-access acceptance to AXI response codes.

6.1.1 Interface description

No parameters.

Signal	Dir.	Width (bits)	Meaning
<code>clk_i</code>	input	1	Rising-edge clock input.
<code>rst_n_i</code>	input	1	Asynchronous reset, active low.
<code>s_axi_awaddr_i</code>	input	32	Write byte address.
<code>s_axi_awvalid_i</code>	input	1	Write address is valid; hold until handshake.
<code>s_axi_awready_o</code>	output	1	Receiver can accept the write address.
<code>s_axi_wdata_i</code>	input	32	Write payload.
<code>s_axi_wstrb_i</code>	input	4	One enable bit per write byte.
<code>s_axi_wvalid_i</code>	input	1	Write data is valid; hold until handshake.
<code>s_axi_wready_o</code>	output	1	Receiver can accept the write data.
<code>s_axi_bresp_o</code>	output	2	Write response: OKAY=00, SLVERR=10, DECERR=11.
<code>s_axi_bvalid_o</code>	output	1	Write response is valid; hold until handshake.
<code>s_axi_bready_i</code>	input	1	Receiver can accept the write response.
<code>s_axi_araddr_i</code>	input	32	Read byte address.
<code>s_axi_arvalid_i</code>	input	1	Read address is valid; hold until handshake.
<code>s_axi_arready_o</code>	output	1	Receiver can accept the read address.
<code>s_axi_rdata_o</code>	output	32	Read payload.
<code>s_axi_rresp_o</code>	output	2	Read response: OKAY=00, SLVERR=10, DECERR=11.
<code>s_axi_rvalid_o</code>	output	1	Read response is valid; hold until handshake.
<code>s_axi_rready_i</code>	input	1	Receiver can accept the read response.
<code>wr_en_o</code>	output	1	Register write pulse after both AW and W have been collected.
<code>wr_addr_o</code>	output	6	Register word offset: captured AWADDR[7:2].
<code>wr_data_o</code>	output	32	Captured write data.
<code>wr_strb_o</code>	output	4	Captured byte-lane enables.
<code>wr_ok_i</code>	input	1	Offset is writable; low selects SLVERR. Peripheral must qualify state updates.

Signal	Dir.	Width (bits)	Meaning
rd_addr_o	output	6	Register word offset ARADDR[7:2].
rd_data_i	input	32	Register read value for rd_addr_o.
rd_ok_i	input	1	Offset is readable; low selects SLVERR.

6.1.2 Functional description

AW and W are captured independently and one register-write pulse is issued after both are available. BRESP is retained until BREADY. An AR handshake samples the peripheral read mux and RRESP, which remain stable until RREADY. Register acceptance selects OKAY or SLVERR.

7 List of bugs and workarounds

No open RTL defects are known at revision 2.2. Limitations that software and follow-up projects have to work around:

Limitation	Workaround
A level request that drops before the claim produces a spurious claim.	Use edge mode for short pulses. The handler reads <code>SRCx_STATUS.SPUR</code> and still sends the EOI.
A software trigger needs <code>WSTRB[3:2]</code> and <code>WSTRB[0]</code> .	Arm it with a 32-bit store; byte and halfword stores cannot set a request.
Lowering <code>NEST_MAX</code> below the current depth keeps the open levels.	Offers resume once EOIs bring the depth below the limit; change <code>NEST_MAX</code> while no source is in service for immediate effect.